Conferencia 8: búsqueda, ordenación y costo

Programación · Ciencia de la Computación y Ciencia de Datos · MatCom · 2026-11-04

La conferencia 6 terminó con una función que encontraba las diez palabras más frecuentes de un capítulo del Quijote. Lo hacía así: buscar la más frecuente, apartarla, buscar la más frecuente de las que quedan, apartarla, y así diez veces. Al final dijimos que eso era una forma de **ordenar**, y que hoy le pondríamos nombre.

Hoy hacemos tres cosas. Primero, **buscar**: decir si un valor está en una lista y dónde. Segundo, **ordenar**: poner los elementos de una lista de menor a mayor, escrito por nosotros, sin `sort` ni `sorted`. Tercero, **contar**: cuánto trabajo hace cada uno de esos programas, y qué le pasa a ese trabajo cuando la lista crece. Esa tercera parte es la importante. Las dos primeras son la excusa.

Python ya trae `in`, `index`, `sort` y `sorted`, y en un programa de verdad se usan esos. Los escribimos a mano por la misma razón por la que en la conferencia 4 escribimos `sum` y `max`: para saber qué hacen por dentro, y porque los algoritmos de hoy son los primeros sobre los que se puede razonar cuánto cuestan.

1. Búsqueda lineal

Dada una lista y un valor, queremos el índice donde está el valor, o `-1` si no está. La manera obvia es mirar los elementos uno por uno:

```python
def buscar(lista, x):
    """Devuelve el primer índice donde está x, o -1 si no está."""
    for i in range(len(lista)):
        if lista[i] == x:
            return i
    return -1

edades = [19, 18, 21, 18, 20, 22]
print(buscar(edades, 21))
print(buscar(edades, 18))
print(buscar(edades, 30))
```

```
2
1
-1
```

El `return` dentro del ciclo es el de la conferencia 3: en cuanto aparece el valor, la función termina, y el ciclo con ella. Solo si el ciclo se acaba sin encontrarlo se llega al `return -1`.

¿Cuánto trabajo hace? Depende de dónde esté el valor. Si está primero, una comparación. Si está último o no está, tantas comparaciones como elementos tenga la lista. Para verlo, contamos:

```python
def buscar_contando(lista, x):
    """Como buscar, pero devuelve también cuántas comparaciones hizo."""
    comparaciones = 0
    for i in range(len(lista)):
        comparaciones += 1
        if lista[i] == x:
            return i, comparaciones
    return -1, comparaciones

numeros = list(range(1_000_000))
print(buscar_contando(numeros, 0))
```

```
print(buscar_contando(numeros, 500_000))
print(buscar_contando(numeros, -5))
```

```
(0, 1)
(500000, 500001)
(-1, 1000000)
```

Cuando el valor no está, hay que mirar **todos** los elementos para estar seguro, porque el que no miraste podía ser el que buscabas. Con una lista de un millón son un millón de comparaciones, y con una de dos millones serían dos millones. El trabajo crece en proporción al tamaño de la lista, y por eso se llama búsqueda **lineal**. Es lo que hace `x in lista` por dentro, y es la razón de que en la conferencia 6 preguntar si una palabra estaba en una lista fuera tan lento comparado con un conjunto.

No se puede hacer mejor sin saber algo más de la lista. Si los elementos están en cualquier orden, cualquier elemento que no mires podría ser el que buscas.

2. Búsqueda binaria

Pero a veces sí sabemos algo más: que la lista está **ordenada**. En la conferencia 2 la computadora adivinaba un número entre 1 y 100 en siete preguntas, partiendo siempre el rango a la mitad. Es la misma idea. Si miro el elemento del medio de una lista ordenada y es menor que lo que busco, todo lo que está a su izquierda también es menor, y puedo descartar esa mitad entera sin mirarla.

Llevamos dos índices, `izq` y `der`, que marcan el pedazo de la lista donde el valor todavía puede estar. Al principio es la lista entera:

```
def buscar_binaria(lista, x):
    """Busca x en una lista ordenada. Devuelve su índice, o -1."""
    izq = 0
    der = len(lista) - 1

    while izq <= der:
        medio = (izq + der) // 2
        if lista[medio] == x:
            return medio
        elif lista[medio] < x:
            izq = medio + 1      # x, si está, está a la derecha de medio
        else:
            der = medio - 1      # x, si está, está a la izquierda de medio

    return -1

ordenada = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
print(buscar_binaria(ordenada, 13))
print(buscar_binaria(ordenada, 2))
print(buscar_binaria(ordenada, 29))
print(buscar_binaria(ordenada, 12))
```

```
5
0
9
-1
```

Haz a mano la búsqueda del 13. Al principio `izq = 0` y `der = 9`, el medio es 4 y `ordenada[4]` es 11, menor que 13: el 13 tiene que estar a la derecha, así que `izq = 5`. Ahora el medio es 7, `ordenada[7]` es 19, mayor: `der = 6`. El medio es 5, `ordenada[5]` es 13, y lo encontramos.

Cuando el valor no está, como el 12, los índices se cruzan: `izq` termina a la derecha de `der`, el pedazo donde podía estar queda vacío y el `while` se acaba.

Ahora contemos:

```python
def buscar_binaria_contando(lista, x):
    """Como buscar_binaria, pero devuelve también cuántas vueltas dio."""
    izq, der = 0, len(lista) - 1
    vueltas = 0
    while izq <= der:
        vueltas += 1
        medio = (izq + der) // 2
        if lista[medio] == x:
            return medio, vueltas
        elif lista[medio] < x:
            izq = medio + 1
        else:
            der = medio - 1
    return -1, vueltas

for n in [1_000, 1_000_000, 1_000_000_000]:
    print(f"{n:>13}: {buscar_binaria_contando(range(n), -5)[1]} vueltas")
```

```
         1000: 9 vueltas
      1000000: 19 vueltas
   1000000000: 29 vueltas
```

Le pasamos `range(n)` en lugar de una lista porque un `range` se indexa igual que una lista ordenada y no ocupa memoria; una lista de mil millones de enteros no cabe en la máquina del laboratorio. Para un millón, la búsqueda lineal hace un millón de comparaciones y la binaria diecinueve vueltas. Multiplicar la lista por mil le añade exactamente diez vueltas, porque cada vuelta parte el pedazo a la mitad, y diez mitades seguidas dividen por $2^{10} = 1024$.

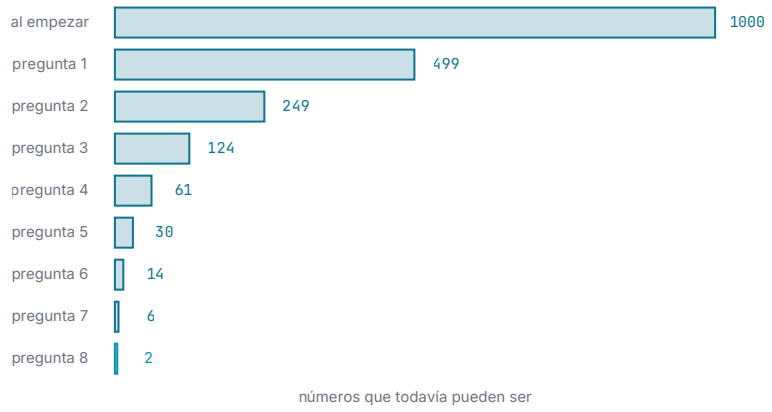


Figura 1. Buscar un valor que no está en una lista ordenada de 1000 elementos. Cada vuelta del ciclo deja vivo, como mucho, la mitad del pedazo anterior.

La cantidad de veces que hay que partir n a la mitad hasta llegar a 1 es el **logaritmo en base 2** de n, que se escribe $\log_2 n$. Es la función inversa de 2^k: $\log_2 1024 = 10$, $\log_2 1\,048\,576 = 20$. Crece tan despacio que, a efectos prácticos, una búsqueda binaria no cuesta nada.

El precio es que la lista tiene que estar ordenada. Si está desordenada, la búsqueda binaria devuelve respuestas falsas sin dar ningún error:

```
print(buscar_binaria([7, 2, 19, 5, 13], 7))
```

```
0
```

El 7 está en la posición 0, y la función dice que no está. Mirar `lista[medio]` y descartar una mitad solo es correcto si la lista está ordenada. Si no lo está, la función hace lo mismo, pero el resultado ya no significa nada.

Un error plantado

La búsqueda binaria es famosa porque es fácil de escribir mal. Esta versión cambia una sola cosa, `izq = medio` en lugar de `izq = medio + 1`, que parece inofensiva: el medio ya se miró, ¿qué más da dejarlo dentro?

```
def buscar_binaria_mal(lista, x):
    izq, der = 0, len(lista) - 1
    while izq <= der:
        medio = (izq + der) // 2
        if lista[medio] == x:
            return medio
        elif lista[medio] < x:
            izq = medio          # antes era medio + 1
        else:
            der = medio - 1
    return -1
```

Con casi todas las pruebas funciona. Pero busca el 9 en `[3, 7]`: `izq = 0`, `der = 1`, el medio es 0, `lista[0]` es 3, menor que 9, así que `izq = 0`. No cambió nada. La vuelta siguiente es idéntica a la anterior, y la función no termina nunca. Lo vemos dando solo seis vueltas:

```
lista, x = [3, 7], 9
izq, der = 0, len(lista) - 1
for vuelta in range(6):
    medio = (izq + der) // 2
    if lista[medio] < x:
        izq = medio
    print(f"vuelta {vuelta}: izq={izq} der={der} medio={medio}")
```

```
vuelta 0: izq=0 der=1 medio=0
vuelta 1: izq=0 der=1 medio=0
vuelta 2: izq=0 der=1 medio=0
vuelta 3: izq=0 der=1 medio=0
vuelta 4: izq=0 der=1 medio=0
vuelta 5: izq=0 der=1 medio=0
```

Lo que garantiza que un `while` termina es que cada vuelta achique algo, y aquí el pedazo `der - izq` tiene que achicarse. Con `medio + 1` y `medio - 1` siempre se achica, porque el medio queda fuera. Con `izq = medio` hay un caso, el de dos elementos, en que no se achica. Las pruebas con listas de un elemento, o buscando valores que están, no lo encuentran. Por eso conviene probar siempre con listas de cero, uno y dos elementos, y con valores menores que todos, mayores que todos y que caen en medio.

3. Ordenación por selección

Si buscar en una lista ordenada es tanto más barato, vale la pena saber ordenar. El primer método es el de la conferencia 6: buscar el menor, ponerlo primero, buscar el menor de los que quedan, ponerlo segundo, y así. Se llama **ordenación por selección**, porque en cada paso se selecciona el menor de lo que falta.

La diferencia con la conferencia 6 es que ahora no apartamos los elementos en otra lista y no borramos nada: los movemos dentro de la misma lista. Después de la pasada `i`, las posiciones `0` a `i` tienen los `i + 1` elementos más pequeños, ya en orden, y el resto está sin ordenar. La pasada `i` busca el menor del resto y lo intercambia con el que está en la posición `i`:

```
def seleccion(lista):
    """Ordena la lista de menor a mayor, modificándola."""
    n = len(lista)
    for i in range(n - 1):
        menor = i
        for j in range(i + 1, n):
            if lista[j] < lista[menor]:
                menor = j
        lista[i], lista[menor] = lista[menor], lista[i]

notas = [4, 2, 5, 3, 5, 2, 3]
seleccion(notas)
print(notas)
```

```
[2, 2, 3, 3, 4, 5, 5]
```

El intercambio `lista[i], lista[menor] = lista[menor], lista[i]` es el desempaquetado de tuplas de la conferencia 4. La función no devuelve nada: modifica la lista que recibió, igual que `sort`, y el cambio se ve desde fuera porque las dos variables nombran la misma lista.

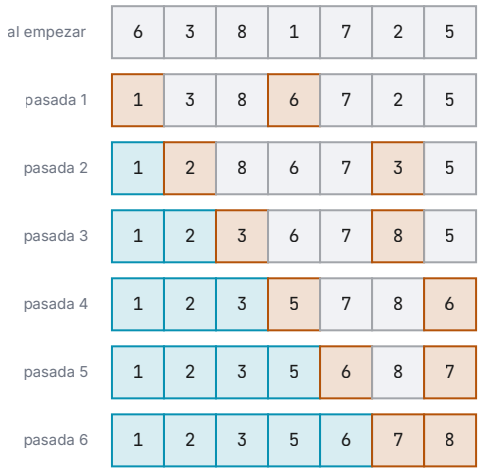


Figura 2. Ordenación por selección de `[6, 3, 8, 1, 7, 2, 5]`. En azul, la parte ya ordenada. En naranja, los dos elementos que intercambió la pasada. Si el menor ya estaba en su sitio, la pasada no intercambia nada.

¿Por qué el ciclo de fuera llega solo hasta `n - 2`? Porque cuando quedan los `n - 1` menores en su sitio, el último elemento es el mayor y ya está en su sitio también.

Ahora contemos las comparaciones `lista[j] < lista[menor]`. La primera pasada compara el primero con los otros $n - 1$. La segunda, $n - 2$. La última, 1. En total:

$$(n-1)+(n-2)+\cdots+2+1=\frac{n(n-1)}{2}.$$

No depende de cómo venga la lista. Aunque ya esté ordenada, la selección hace todas esas comparaciones, porque para saber que el primero es el menor tiene que compararlo con todos los demás. Para $n = 1000$ son casi medio millón.

4. Ordenación por inserción

El segundo método es el que usa casi todo el mundo para ordenar las cartas que tiene en la mano. Tomas las cartas una a una, y cada carta nueva la metes en su lugar entre las que ya tienes, que están ordenadas.

En la lista, después de la pasada `i` las posiciones `0` a `i` están ordenadas entre sí, aunque no sean todavía las más pequeñas de la lista. La pasada `i` toma el elemento de la posición `i` y lo va corriendo hacia la izquierda mientras el de su izquierda sea mayor:

```
def insercion(lista):
    """Ordena la lista de menor a mayor, modificándola."""
    for i in range(1, len(lista)):
        x = lista[i]
        j = i - 1
        while j >= 0 and lista[j] > x:
            lista[j + 1] = lista[j]      # corre el mayor un lugar a la derecha
            j -= 1
        lista[j + 1] = x                 # y x cae en el hueco

notas = [4, 2, 5, 3, 5, 2, 3]
insercion(notas)
print(notas)
```

```
[2, 2, 3, 3, 4, 5, 5]
```

El `while` tiene dos condiciones, y el orden importa. Si `j` llega a `-1`, la primera es falsa y el `and` ya no evalúa la segunda, así que nunca se mira `lista[-1]`, que en Python no daría error sino el último elemento de la lista, y el resultado sería falso sin ningún aviso.

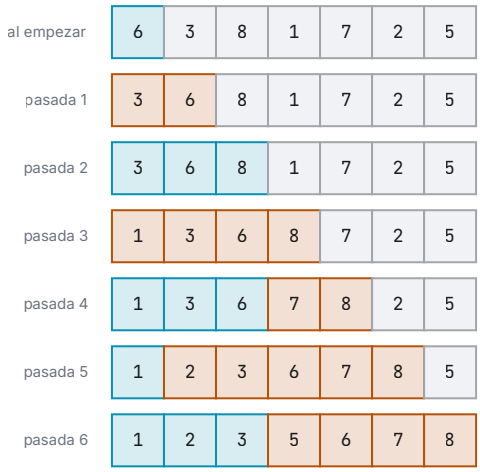


Figura 3. Ordenación por inserción de la misma lista. En azul, el pedazo ordenado entre sí. En naranja, lo que se movió para hacerle hueco al elemento nuevo. Compárala con la figura de la selección: aquí el azul no son todavía los menores de la lista.

Compara la figura 3 con la figura 2. En la selección, la parte azul ya es definitiva: esos elementos no se vuelven a mover. En la inserción, la parte azul está ordenada, pero cualquier elemento que llegue después puede meterse en medio.

El trabajo de la inserción depende de la lista. Si ya está ordenada, cada elemento se compara con el de su izquierda, ve que es menor, y no se mueve: $n - 1$ comparaciones en total. Si está al revés, cada elemento nuevo es menor que todos los anteriores y tiene que recorrerlos todos, y salen otra vez $\frac{n(n-1)}{2}$. Lo comprobamos:

```python
def insercion_contando(lista):
    """Ordena la lista y devuelve cuántas comparaciones hizo."""
    comparaciones = 0
    for i in range(1, len(lista)):
        x = lista[i]
        j = i - 1
        while j >= 0:
            comparaciones += 1
            if lista[j] <= x:
                break
            lista[j + 1] = lista[j]
            j -= 1
        lista[j + 1] = x
    return comparaciones

n = 1000
random.seed(8)
al_azar = [random.randint(1, 10_000) for _ in range(n)]

print("ordenada:   ", insercion_contando(list(range(n))))
print("al revés:   ", insercion_contando(list(range(n, 0, -1))))
print("al azar:    ", insercion_contando(list(al_azar)))
print("n(n-1)/2:   ", n * (n - 1) // 2)
```

```
ordenada:    999
al revés:    499500
al azar:     248214
n(n-1)/2:    499500
```

Para contar hubo que cambiar la forma del ciclo: la comparación `lista[j] > x` pasó del `while` a un `if` con `break`, para poder sumar uno cada vez que se hace. Lo que hace la función es lo mismo.

Con una lista al azar, la inserción hace alrededor de la mitad del peor caso. Tiene sentido: en promedio, cada elemento nuevo tiene la mitad de los anteriores a su derecha. La mitad de $\frac{n(n-1)}{2}$ sigue siendo una cantidad que se multiplica por cuatro cuando n se duplica, y eso es lo que importa, como veremos en la sección 6.

5. Ordenación por intercambio

El tercer método solo mira elementos vecinos. Recorre la lista comparando cada elemento con el siguiente, y si están al revés los intercambia. Al final de una pasada, el mayor de todos ha ido arrastrándose hasta el último lugar. Se repite con el resto hasta que una pasada no tenga nada que intercambiar. Se le llama también **burbuja**, porque los elementos grandes suben hacia el final como burbujas.

```python
def burbuja(lista):
    """Ordena la lista de menor a mayor, modificándola."""
    n = len(lista)
    for k in range(n - 1):
        hubo_cambio = False
```

```
        for j in range(n - 1 - k):
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
                hubo_cambio = True
        if not hubo_cambio:
            break

notas = [4, 2, 5, 3, 5, 2, 3]
burbuja(notas)
print(notas)
```

```
[2, 2, 3, 3, 4, 5, 5]
```

El ciclo de dentro llega cada vez un lugar menos lejos, `n - 1 - k`, porque después de `k` pasadas los `k` mayores ya están al final. La bandera `hubo_cambio` permite parar antes: si una pasada entera no intercambió nada, la lista ya está ordenada. Con una lista ordenada, la burbuja hace una sola pasada de $n - 1$ comparaciones. En el peor caso vuelven a ser $\frac{n(n-1)}{2}$.

Antes de seguir, una costumbre que vale para cualquier algoritmo que escribas: probarlo contra uno que sepas que está bien. Aquí ese uno es `sorted`. Probamos los tres métodos con cientos de listas al azar de tamaños pequeños, que es donde están los casos raros, incluidas la lista vacía y la de un elemento:

```
random.seed(1)
for prueba in range(500):
    n = random.randint(0, 12)
    original = [random.randint(1, 5) for _ in range(n)]
    for metodo in [seleccion, insercion, burbuja]:
        copia = list(original)
        metodo(copia)
        if copia != sorted(original):
            print("FALLA", metodo.__name__, original, copia)

print("pruebas terminadas")
```

```
pruebas terminadas
```

Los números van del 1 al 5 a propósito, para que haya muchos repetidos, que es otro caso que se rompe con facilidad.

6. Contar en lugar de cronometrar

Tenemos tres métodos que ordenan. ¿Cuál es mejor? La respuesta tentadora es cronometrarlos, y lo haremos en la sección siguiente, pero primero contamos operaciones. El reloj depende de la máquina, de lo que esté haciendo la máquina en ese momento y de la versión de Python. La cantidad de comparaciones depende solo del algoritmo y de la lista.

Ya lo hicimos una vez. En la conferencia 4, al triplicar el tope, la criba de Eratóstenes triplicaba sus tachones y el método de dividir uno por uno multiplicaba sus divisiones por casi cinco. Hoy hacemos lo mismo duplicando el tamaño de la lista:

```
def seleccion_contando(lista):
    comparaciones = 0
    n = len(lista)
    for i in range(n - 1):
        menor = i
        for j in range(i + 1, n):
```

```
            comparaciones += 1
            if lista[j] < lista[menor]:
                menor = j
        lista[i], lista[menor] = lista[menor], lista[i]
    return comparaciones

def burbuja_contando(lista):
    comparaciones = 0
    n = len(lista)
    for k in range(n - 1):
        hubo_cambio = False
        for j in range(n - 1 - k):
            comparaciones += 1
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
                hubo_cambio = True
        if not hubo_cambio:
            break
    return comparaciones

random.seed(8)
print(f"{'n':>5} {'lineal':>7} {'binaria':>8} {'selección':>10} "
      f"{'inserción':>10} {'burbuja':>10}")
for n in [100, 200, 400, 800, 1600]:
    lista = [random.randint(1, 10_000) for _ in range(n)]
    lineal = buscar_contando(lista, -5)[1]
    binaria = buscar_binaria_contando(sorted(lista), -5)[1]
    sel = seleccion_contando(list(lista))
    ins = insercion_contando(list(lista))
    bur = burbuja_contando(list(lista))
    print(f"{n:>5} {lineal:>7} {binaria:>8} {sel:>10} {ins:>10} {bur:>10}")
```

```
    n  lineal  binaria  selección  inserción    burbuja
  100     100        6       4950       2573       4947
  200     200        7      19900      10180      19809
  400     400        8      79800      39218      79547
  800     800        9     319600     160691     318610
 1600    1600       10    1279200     639673    1278672
```

Lee la tabla por columnas, de arriba abajo. Cada fila tiene el doble de elementos que la anterior.

- La **búsqueda lineal** duplica su trabajo cuando la lista se duplica. Su trabajo es proporcional a n.
- La **búsqueda binaria** suma una vuelta cuando la lista se duplica. Su trabajo es proporcional a $\log_2 n$.
- Los **tres métodos de ordenación** multiplican su trabajo por cuatro cuando la lista se duplica. Su trabajo es proporcional a n^2. La selección hace exactamente $\frac{n(n-1)}{2}$; la inserción, alrededor de la mitad; la burbuja, algo menos que la selección. Pero los tres se multiplican por cuatro.

Ese "proporcional a" es lo que se llama el **orden de crecimiento** de un algoritmo, y se escribe con una O mayúscula: la búsqueda lineal es $O(n)$, la binaria es $O(\log n)$ y los tres métodos de ordenación son $O(n^2)$. La definición precisa es materia de Estructuras de Datos. Para nosotros, el orden de crecimiento es la respuesta a una pregunta: si la entrada se duplica, ¿por cuánto se multiplica el trabajo? Por 1 y poco, por 2, o por 4.

Las constantes, como el $\frac{1}{2}$ de $\frac{n(n-1)}{2}$, deciden cuál de dos métodos del mismo orden es más rápido. El orden de crecimiento decide algo más importante: hasta qué tamaño de entrada el programa sirve.

7. El reloj

Ahora sí, el reloj. Medimos el tiempo de los tres métodos sobre las mismas listas al azar, duplicando el tamaño:

```python
def cronometrar(metodo, lista):
    """Segundos que tarda metodo en ordenar una copia de la lista."""
    copia = list(lista)
    inicio = time.perf_counter()
    metodo(copia)
    return time.perf_counter() - inicio

random.seed(8)
tamanos = [500, 1000, 2000, 4000]
tiempos = {"selección": [], "inserción": [], "burbuja": []}

for n in tamanos:
    lista = [random.randint(1, 1_000_000) for _ in range(n)]
    tiempos["selección"].append(cronometrar(seleccion, lista))
    tiempos["inserción"].append(cronometrar(insercion, lista))
    tiempos["burbuja"].append(cronometrar(burbuja, lista))

print(f"{'n':>5} {'selección':>10} {'inserción':>10} {'burbuja':>10}")
for k, n in enumerate(tamanos):
    print(f"{n:>5} {tiempos['selección'][k]:>9.3f}s {tiempos['inserción'][k]:>9.3f}s "
          f"{tiempos['burbuja'][k]:>9.3f}s")
```

```
    n  selección  inserción    burbuja
  500     0.005s     0.005s     0.011s
 1000     0.017s     0.018s     0.037s
 2000     0.071s     0.074s     0.163s
 4000     0.311s     0.327s     0.793s
```

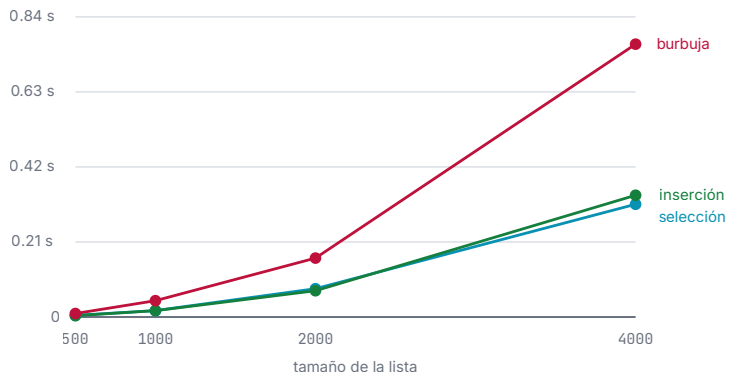


Figura 4. Los segundos de la tabla anterior. Las tres curvas se doblan hacia arriba: cada vez que la lista se duplica, el tiempo se multiplica por cuatro.

La burbuja es la más lenta, aunque haga menos comparaciones que la selección. Hace muchos más intercambios, y un intercambio cuesta más que una comparación. Las constantes, otra vez. Pero las tres curvas tienen la misma forma, la de figura 4, y esa forma es la del n^2. Lo vemos dividiendo cada tiempo por el anterior:

```python
for metodo, ts in tiempos.items():
    razones = [ts[k + 1] / ts[k] for k in range(len(ts) - 1)]
    print(f"{metodo:>10}: " + "  ".join(f"×{r:.1f}" for r in razones))
```

```
 selección: ×4.0  ×3.3  ×4.5
 inserción: ×3.4  ×4.5  ×4.1
   burbuja: ×3.6  ×3.8  ×4.1
```

Las razones no salen exactamente 4, porque el reloj mide también otras cosas, pero rondan el 4. Con eso podemos hacer una predicción. Si 4000 elementos tardan lo que dice la tabla, un millón son 250 veces más elementos, y por tanto unas $250^2 = 62\,500$ veces más tiempo:

```
for metodo, ts in tiempos.items():
    estimado = ts[-1] * (1_000_000 / 4000) ** 2
    print(f"{metodo:>10}: unas {estimado / 3600:.1f} horas para un millón")
```

```
 selección: unas 6.0 horas para un millón
 inserción: unas 6.1 horas para un millón
   burbuja: unas 13.3 horas para un millón
```

Horas, para un millón de números. Y un millón no es una cantidad rara: La Habana tiene más habitantes que eso. Y no se arregla con una computadora más rápida. Una diez veces más rápida pasa de horas a decenas de minutos, y con diez millones de elementos el problema vuelve, multiplicado por cien.

Para que se vea que no es un problema sin solución, esto es lo que tarda `sorted`, la función de Python, con ese mismo millón:

```
random.seed(8)
millon = [random.randint(1, 1_000_000) for _ in range(1_000_000)]
inicio = time.perf_counter()
ordenado = sorted(millon)
print(f"sorted: {time.perf_counter() - inicio:.2f} s")
```

```
sorted: 0.38 s
```

Menos de un segundo. `sorted` no es un n^2 escrito en un lenguaje más rápido; es otro algoritmo, con otro orden de crecimiento. Cómo se ordena un millón de números en un segundo es el tema de la conferencia 10.

8. El proyecto: las palabras por frecuencia

Volvemos al Quijote. En la conferencia 6 contamos las palabras del primer capítulo con un diccionario. Ahora queremos **todas** las palabras ordenadas de la más frecuente a la menos frecuente, escritas con un método nuestro.

El diccionario da parejas `(palabra, veces)`. Para ordenar parejas por su segundo elemento, y de mayor a menor, basta cambiar la comparación de la inserción:

```
def ordenar_por_frecuencia(parejas):
    """Ordena parejas (palabra, veces) de más a menos frecuente, modificando la lista."""
    for i in range(1, len(parejas)):
        x = parejas[i]
        j = i - 1
        while j >= 0 and parejas[j][1] < x[1]:
            parejas[j + 1] = parejas[j]
            j -= 1
        parejas[j + 1] = x

conteo = frecuencias(palabras_de("quijote-cap1.txt"))
parejas = list(conteo.items())
ordenar_por_frecuencia(parejas)

print(len(parejas), "palabras distintas")
```

```
for palabra, veces in parejas[:8]:
    print(f"  {palabra:<8} {veces:3}")
```

```
719 palabras distintas
  de       122
  y        106
  que       89
  a         44
  el        40
  en        38
  la        35
  su        35
```

Lo único que cambió es la condición del `while`: `parejas[j][1] < x[1]` compara los conteos, y el `<` en lugar de `>` pone primero los mayores. El resto de la función es la inserción de la sección 4 línea por línea. Un algoritmo de ordenación no sabe qué está ordenando; solo necesita saber cuándo un elemento va antes que otro.

Hay un detalle que vale la pena notar. Las palabras con el mismo conteo quedan en el mismo orden en que estaban en el diccionario, porque la inserción nunca pasa un elemento por encima de otro igual: el `while` se detiene ante uno que no es estrictamente menor. Un método con esa propiedad se llama **estable**, y la tiene también `sorted`. La selección no la tiene; el ejercicio 4 te pide encontrar una lista que lo demuestre.

La segunda parte del proyecto es buscar. Queremos un programa al que le das una palabra y te dice cuántas veces aparece. Con el diccionario de la conferencia 6 eso ya sale directo, pero vamos a hacerlo con lo de hoy para ver la búsqueda binaria trabajando: ordenamos las palabras distintas alfabéticamente, y buscamos en esa lista.

```
alfabetico = list(conteo)
insercion(alfabetico)
print(alfabetico[:6])
print(alfabetico[-4:])

for palabra in ["hidalgo", "rocinante", "molino"]:
    i = buscar_binaria(alfabetico, palabra)
    if i == -1:
        print(f"{palabra}: no aparece")
    else:
        print(f"{palabra}: posición {i}, aparece {conteo[palabra]}")
```

```
['a', 'acabar', 'acabándolos', 'acomodada', 'acomodársele', 'acontece']
['él', 'ésta', 'ídolo', 'ínsula']
hidalgo: posición 337, aparece 4
rocinante: posición 585, aparece 1
molino: no aparece
```

La inserción ordena las cadenas sin cambiar nada, porque `<` entre cadenas compara en orden alfabético, letra por letra, como en un diccionario de papel. Ese orden es el de los códigos de los caracteres, y en esos códigos las letras con tilde y la `ñ` van después de la `z`: por eso `él` e `ínsula` salen al final. Si una palabra empieza por `á`, la búsqueda binaria la encuentra igual, porque la lista y la búsqueda usan la misma comparación.

9. Resumen

- La **búsqueda lineal** mira los elementos uno por uno. En el peor caso mira todos: su trabajo es proporcional a n.
- La **búsqueda binaria** descarta la mitad del pedazo en cada vuelta. Necesita la lista ordenada, y si no lo está devuelve respuestas falsas sin ningún error. Su trabajo es proporcional a $\log_2 n$: unas veinte vueltas para un millón.
- En la búsqueda binaria, `izq = medio + 1` y `der = medio - 1` garantizan que el pedazo se achica en cada vuelta. Cambiar uno de los dos puede dejar el ciclo dando vueltas para siempre en listas de dos elementos.
- La **selección** busca el menor de lo que falta y lo intercambia a su lugar. Hace siempre $\frac{n(n-1)}{2}$ comparaciones.
- La **inserción** mete cada elemento nuevo en su lugar dentro de la parte ya ordenada. Es rápida con listas casi ordenadas y lenta con listas al revés. Es estable.
- La **burbuja** intercambia vecinos que están al revés, y para cuando una pasada no cambia nada.
- Los tres métodos son de orden n^2: duplicar la lista multiplica su trabajo por cuatro. Para un millón de elementos tardan horas.
- Contar operaciones dice cómo crece el trabajo sin depender de la máquina. El reloj lo confirma.
- Todo algoritmo nuevo se prueba contra uno que se sabe correcto, con muchas entradas pequeñas al azar, incluidas la vacía y la de un elemento.

Ejercicios

1. Escribe `buscar_ultimo(lista, x)` que devuelva el **último** índice donde aparece `x`, o `-1`. Hazlo de dos maneras: recorriendo desde el principio, y recorriendo desde el final. ¿Cuál hace menos comparaciones, y en qué casos?
2. La búsqueda binaria de la sección 2 devuelve **algún** índice donde está `x`, no necesariamente el primero. Encuentra una lista con repetidos donde no devuelva el primero. Después escribe `buscar_binaria_primero(lista, x)` que siempre devuelva el primero, sin dejar de partir a la mitad: cuando encuentres `x`, no termines, sigue buscando en la mitad izquierda.
3. Escribe `insertar_ordenado(lista, x)` que reciba una lista ordenada y le agregue `x` en su lugar, de manera que siga ordenada. Usa la búsqueda binaria para encontrar el lugar. ¿Cuántas comparaciones hace? ¿Y cuántos elementos tiene que mover?
4. Encuentra una lista de parejas `(nombre, nota)` donde ordenar por nota con la selección cambie el orden de dos estudiantes con la misma nota. Es decir, demuestra que la selección no es estable. ¿Puedes arreglarla?
5. Modifica la selección para que en cada pasada busque a la vez el menor y el mayor de lo que falta, y los ponga uno al principio y otro al final. ¿Cuántas pasadas hace? ¿Cuántas comparaciones? ¿Cambia el orden de crecimiento?
6. Cuenta, además de las comparaciones, cuántas veces se **escribe** en la lista en cada uno de los tres métodos, para listas al azar de 100, 200 y 400 elementos. ¿Cuál escribe menos? Con eso explica por qué la burbuja es más lenta en el reloj que la selección aunque haga menos comparaciones.
7. Construye una lista de 1000 elementos con la que la inserción haga **menos** de 2000 comparaciones, y otra con la que la burbuja haga exactamente 999. Después construye una lista de 1000 elementos casi ordenada, en la que solo el último elemento esté fuera de su lugar: es el menor de todos. ¿Cuántas pasadas hace la burbuja? ¿Y la inserción?
8. Sin usar `sorted`, `sort` ni ningún método de los de hoy, escribe `esta_ordenada(lista)` que diga si una lista está ordenada de menor a mayor. ¿Cuál es su orden de crecimiento? Úsala para que

`buscar_binaria` dé un error claro cuando la lista no está ordenada. ¿Qué pasa con el costo de la búsqueda binaria si hace esa comprobación cada vez que se la llama?